

Introduction

1. Install .NET SDK

2. VS Code Extensions

3. Create C# Project

4. Run & Debug

Windows Forms

Shortcuts

C#.NET Setup Guide for VS Code

A comprehensive dark Microsoft Purple themed workflow blueprint.

Configure a cross-platform C# environment within Visual Studio Code by establishing the underlying compiler runtime, language tools, and deployment scripts.

1. Download and Install the .NET SDK

Your workspace requires compiler binaries to parse code execution structures.

- Get the latest stable SDK from the official [Microsoft .NET Download Page](#).
- Run the system package installer and finish the default setup process.

Verify Installation

Open a terminal windows environment and run the check string:

```
dotnet --version
```

2. Install the VS Code

Introduction

1. Install .NET SDK

2. VS Code Extensions

3. Create C# Project

4. Run & Debug

Windows Forms

Shortcuts

Extensions

Integrate compilation engines and debug features directly into the editor view footprint.

1. Open Visual Studio Code.
2. Go to the Extensions layout icon menu panel (or press `Ctrl+Shift+X` / `Cmd+Shift+X`).
3. Search for `"C# Dev Kit"` and click Install.

System Note: Installing the C# Dev Kit bundle container automatically sets up base debugger syntax rules and background runtime tracking tools.

3. Create a New C# Project

Choose an execution architecture deployment path below to generate your program boilerplate structures safely.

Option A: The GUI Command Interface

1. Open the Command Palette using `Ctrl+Shift+P` (or `Cmd+Shift+P` on Mac).
2. Select `.NET: New Project...` from the dropdown options list.
3. Choose the `Console App` template layout object.
4. Select a folder workspace on your local drive disk.
5. Type your project name (e.g., `MyFirstApp`) and press Enter.

Option B: Integrated Terminal Scripting

Initialize project assets rapidly via direct shell scripts. Launch your terminal panel window layout (`Ctrl+``)

.NET ENVIRONMENT

Introduction

1. Install .NET SDK

2. VS Code Extensions

3. Create C# Project

4. Run & Debug

Windows Forms

Shortcuts

and process these commands sequence lines sequentially:

```
mkdir MyFirstApp
cd MyFirstApp
dotnet new console
```

Open the workspace files safely by going to the top menu and clicking **File > Open Folder**, then select your new directory choice.

4. Run and Debug Your Code

Locate the `Program.cs` file inside the root workspace folder view list to inspect your primary logic block structure layout.

Terminal Execution

Run the direct compilation command tool inside the active terminal container framework:

```
dotnet run
```

Editor UI Execution

Click the visual Play Button interface icon located on the top header ribbon, or press `Ctrl+F5` (`Cmd+F5`).

Introduction

1. Install .NET SDK

2. VS Code Extensions

3. Create C# Project

4. Run & Debug

Windows Forms

Shortcuts

Debugging Operations

Switch to the Run and Debug icon panel, choose C# as the environment runtime engine, and place break markers.

Windows Forms (WinForms) Support

VS Code runs WinForms logic fine on Windows, but does not feature an official visual drag-and-drop designer window palette tool.

Scaffolding a WinForms Project Matrix

```
dotnet new winforms -o MyWinFormsApp
cd MyWinFormsApp
dotnet run
```

UI Build Design Methodologies

Development Path	Workflow Execution Paradigm
Code-First Composition	Instantiate and format buttons, text layout forms, grids, and canvas logic manually inside code modules like Form1.cs.
Community Extensions	Incorporate unofficial experimental add-ons such as the C# Form GUI Editor grid array via the marketplace layout hub.

.NET ENVIRONMENT

Introduction

1. Install .NET SDK

2. VS Code Extensions

3. Create C# Project

4. Run & Debug

Windows Forms

Shortcuts

Development
Path

Workflow Execution Paradigm

**Visual Studio
IDE Integration**

Migrate your working solution to the full Visual Studio Community Edition tool for optimized visual toolboxes.

Useful Environment Shortcuts

Command Palette

`Ctrl + Shift + P`

Extensions Panel

`Ctrl + Shift + X`

Toggle Terminal

`Ctrl + ``

Build Project

`dotnet build`